

RAG agéntico y evaluación con RAGAS.

Ya construimos un RAG que recupera bien. Hoy le damos herramientas para actuar cuando hacen falta, y aprendemos a evaluarlo con RAGAS (mira el contexto y no solo la respuesta).

DOCENTE: Manuela Larrea Gomez

TOOL USE

ReAct

RAGAS

La sesión de hoy.



Recap: dónde estamos y el bucle agentic



Tool use y function calling  Lab A



ReAct y cuándo (no) usar un agente  Lab B



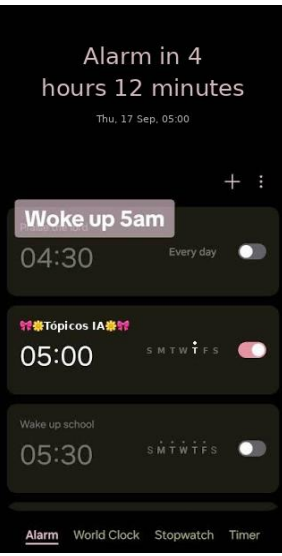
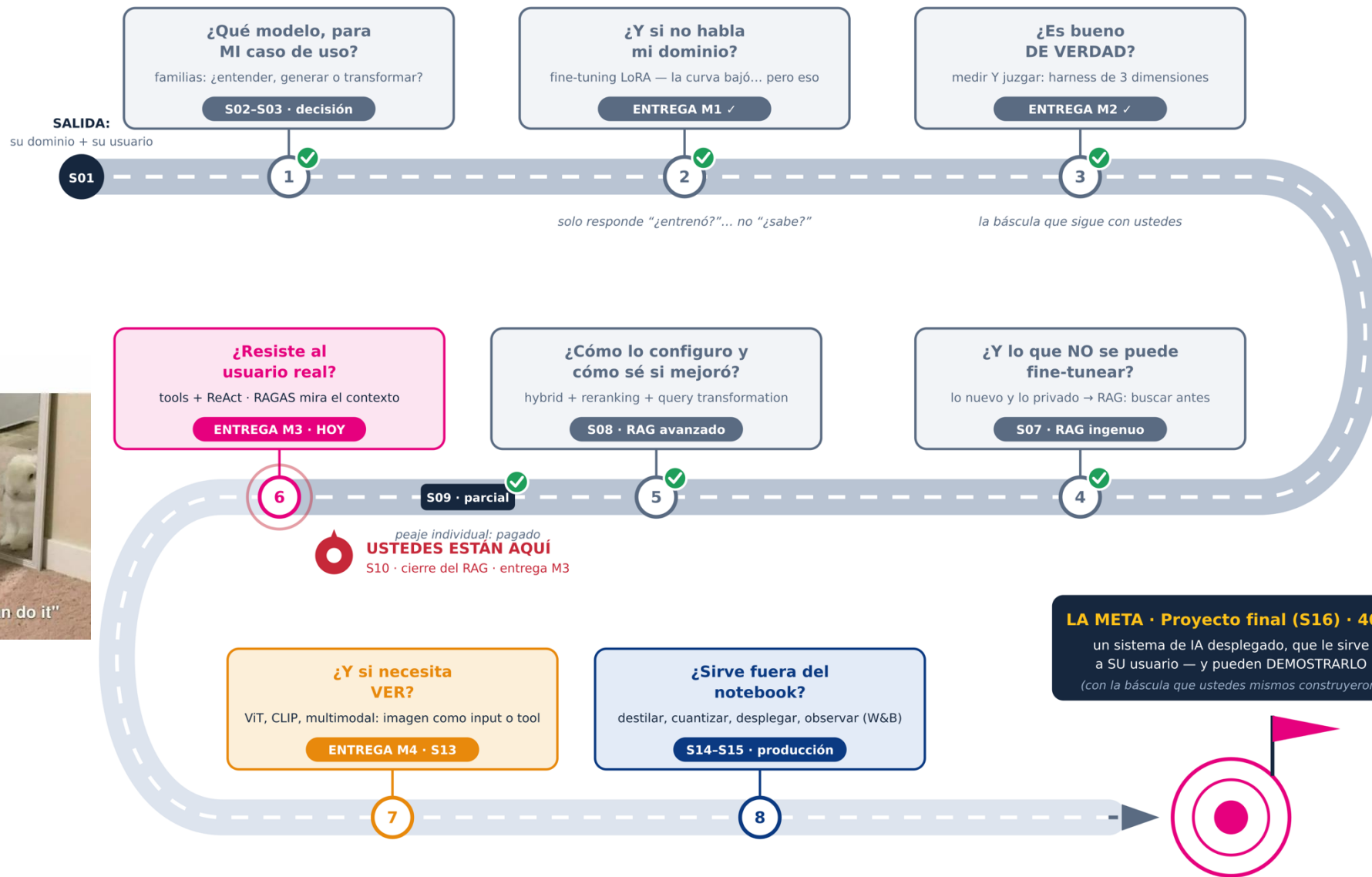
RAGAS: faithfulness, context precision/recall, answer relevancy  Lab C



Cerrar el sistema y hacer la entrega



Profundización y lo que viene (visión, S11)



Lo que falta: **M3 hoy (15%)** · M4 en S13 (10%) · despliegue en S15 · **proyecto final en S16 (40%)** = M1+M2+M3+M4 + producción

Calendario de entregas · 2026-2

SI4006 · Tópicos Especiales y Aplicaciones en IA · Universidad EAFIT

COMO LEERLO

● Sesión de clase

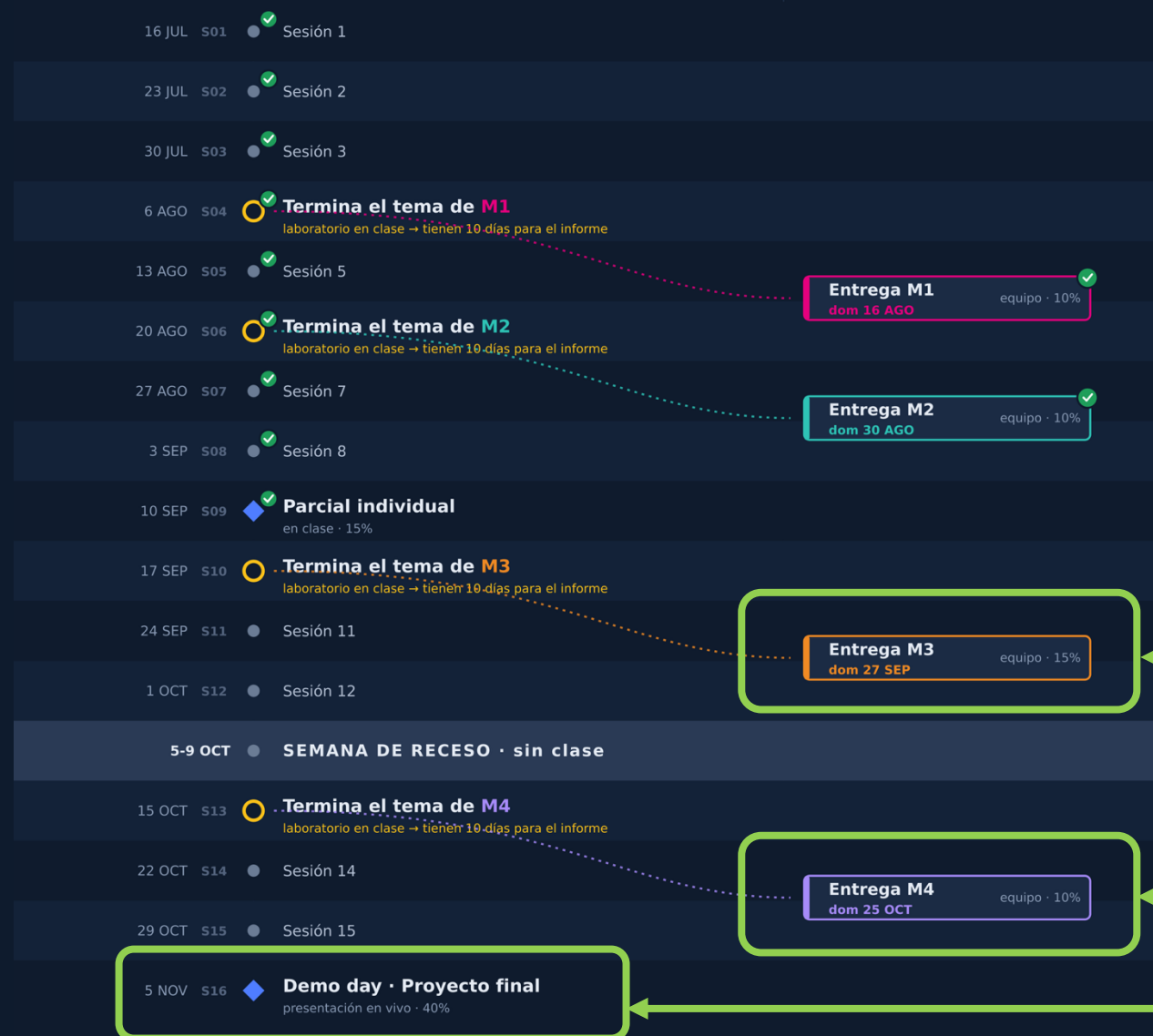
○ Termina el tema de una entrega

■ Cierre de entrega (domingo)

◆ Parcial / Demo en vivo

CIERRA (domingo 23:59)

02 / 32



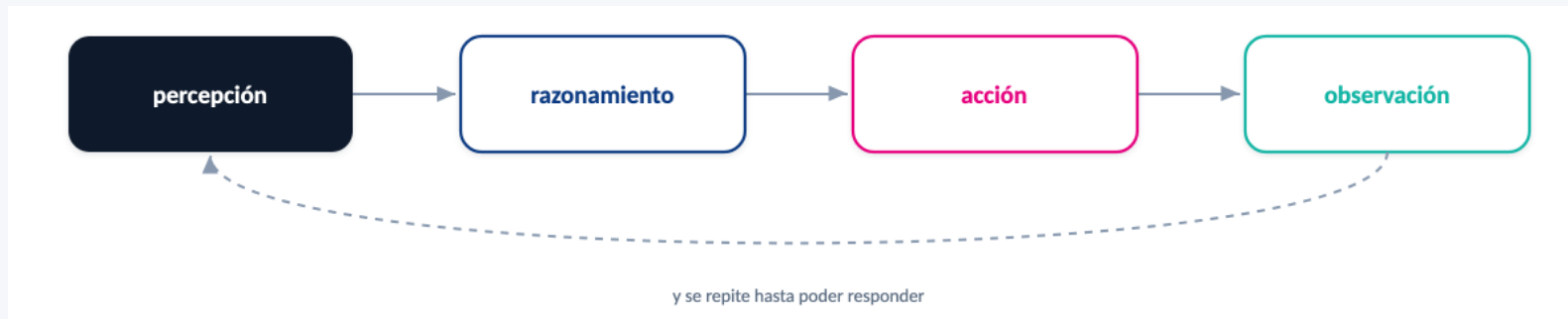
Cada entrega de equipo se construye en el laboratorio de su sesión y cierra el domingo siguiente. El parcial y el demo day son en vivo.

Las entregas son acumulativas: cada una es una capa del proyecto final.

Agentes: percibir, razonar, actuar.

Recordemos dónde estamos, y el valor de darle acciones al modelo.

Dónde estamos, y qué sigue.



Ya tenemos un RAG avanzado que recupera bien, pero sus respuestas están limitadas a una iteración.

Vamos a darle la opción de **actuar**, llamando a una herramienta cuando se requiera, y repetir hasta poder responder apropiadamente.

Tool use: el modelo llama funciones.

Le damos herramientas y él decide cuándo usarlas.

El modelo decide cuándo y qué invocar

SIN HERRAMIENTAS



Responde con lo que sabe

Contesta solo con su conocimiento y el contexto recuperado. Si necesita un cálculo exacto o un dato en vivo, tiende a inventarlo.

CON HERRAMIENTAS



Puede actuar

Le damos funciones (una calculadora, una búsqueda, una API). En vez de inventar, el modelo solicita que la función se ejecute y usa el resultado.

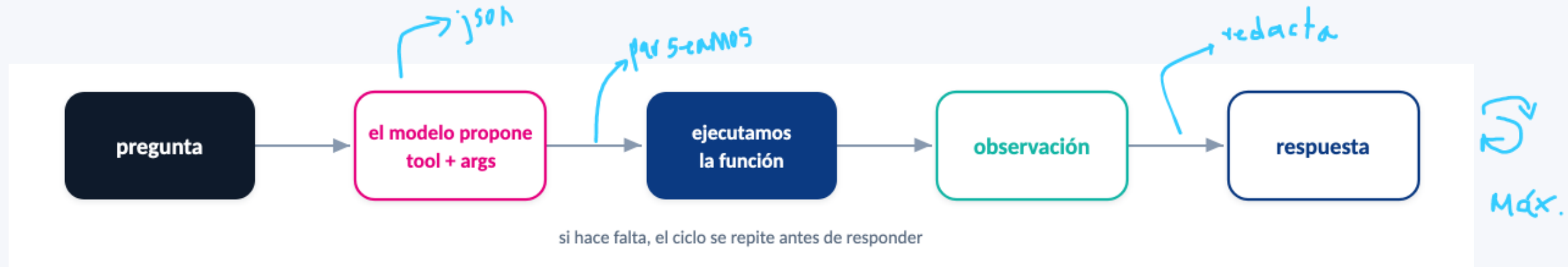
El modelo no ejecuta las funciones, propone la llamada y “nosotros” ejecutamos la función y le devolvemos el resultado para que trabaje.

Una herramienta se describe con un esquema.

```
{  
  "name": "calculadora",  
  "description": "Evalúa una expresión aritmética y devuelve el número.",  
  "parameters": { "expresion": "string" }  
}
```

El modelo lee ese esquema y, determina si requiere el uso de la herramienta, responde con su nombre y los argumentos en JSON.

Cómo funciona una llamada a herramienta.



El modelo devuelve un JSON con la herramienta y sus argumentos. Lo parseamos, ejecutamos la función y le pasamos el resultado como **observación**.

Con esa observación, el modelo redacta la respuesta final (o pide otra herramienta).

Al notebook · Lab A

Function calling: una calculadora y una tool de dominio

Definimos dos herramientas con su esquema, le damos la pregunta al modelo y dejamos que decida si llamar una o responder directo. Ejecutamos la llamada, le devolvemos el resultado como observación y vemos la respuesta final.

```
definir_tools → el modelo propone → ejecutar → observación → respuesta ·  
S10_Lab_Agentic_RAG_RAGAS.ipynb
```

Qué vimos con las herramientas.

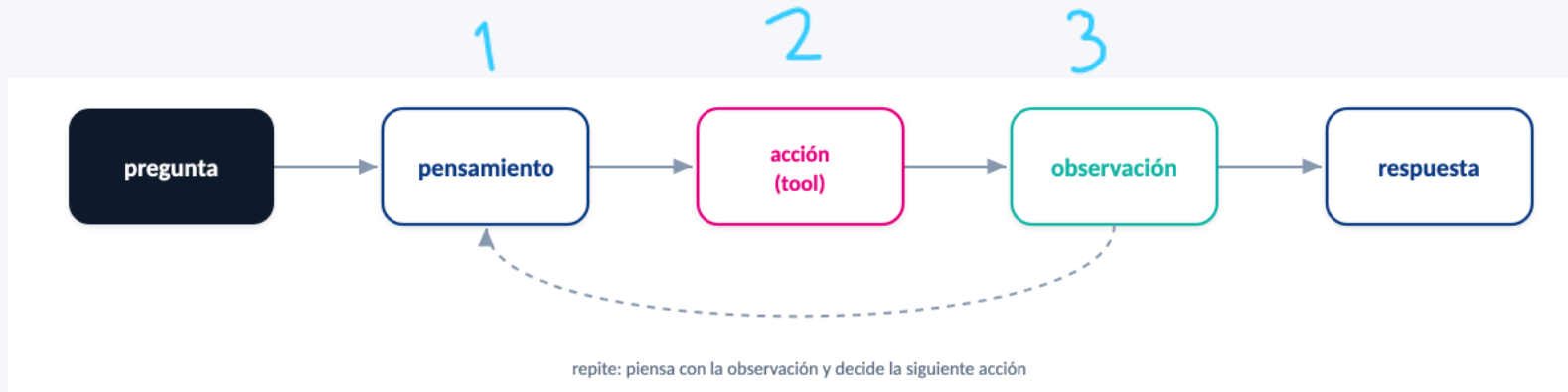
- **El modelo eligió la herramienta** ante un cálculo exacto pidió la calculadora, en vez de estimar el número.
- **Nosotros ejecutamos** el modelo solo propone; la función corre de nuestro lado y le devolvemos el resultado.
- **El JSON hay que validarlo** a veces propone argumentos mal formados, así que conviene parsear con cuidado.

ReAct:

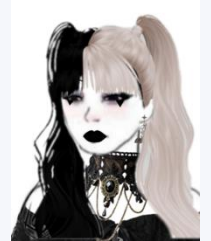
pensar, actuar, observar.

El modelo alterna razonamiento y acciones, y repite hasta resolver.

Razonar y actuar, en un ciclo.



ALTERNA



En ReAct el modelo escribe un **pensamiento**, elige una **acción** (llamar una herramienta), lee la **observación** y sigue.

Así resuelve preguntas que necesitan varios pasos, no solo una recuperación.

El retrieval también es una herramienta.



En vez de recuperar siempre, el agente decide **cuándo** buscar en el corpus.

Una pregunta simple la responde directo; una que necesita el documento, la busca. El RAG avanzado de la S08 se vuelve una herramienta más.

No todo necesita **un agente**.

UN AGENTE AYUDA CUANDO

La pregunta necesita varios pasos, o decidir entre fuentes, o un cálculo o un dato en vivo que el modelo no debe inventar.

UN AGENTE ESTORBA CUANDO

Una sola pasada de RAG ya responde. Más pasos son más latencia, más costo y más formas de fallar.

Regla práctica: empiecen con el RAG de una pasada y agreguen agente solo si el harness mejora de verdad.

Al notebook · Lab B

Un mini-agente ReAct con RAG y calculadora

Montamos un bucle ReAct sencillo: el modelo piensa, elige entre buscar en el corpus o calcular, lee la observación y repite hasta responder. Probamos una pregunta de un paso y una de varios, y las comparamos con el RAG de una sola pasada.

`pensamiento → acción → observación → repetir → respuesta`

Qué probamos con el agente.

- **El agente encadenó pasos** en la pregunta compuesta buscó y luego calculó; el RAG de una pasada no podía.
- **En la simple, sobró** para la pregunta directa dio más vueltas sin mejorar la respuesta.
- **Más pasos, más latencia** cada paso es una generación más, así que el agente cuesta bastante más tiempo.

RAGAS: evaluar el RAG por dentro.

El harness mira la respuesta; RAGAS también mira el contexto que usamos.

El harness no ve el contexto.

Nuestro harness compara la respuesta con lo esperado, y eso está bien. Pero en un RAG importan dos cosas más: ¿el contexto que recuperamos era el correcto?, y ¿la respuesta de verdad se apoya en ese contexto, o lo ignoró? RAGAS mide justo eso.

La idea clave: **una respuesta puede acertar por la razón equivocada** (el modelo ya lo sabía) o fallar por un contexto pobre. RAGAS separa esos casos.

Cuatro métricas, dos preguntas.

1 • EL SISTEMA RESOLVIÓ EL CASO (esto todavía no es RAGAS)



2 • RAGAS le pone cuatro notas de 0 a 1 sobre ese caso.

Cada nota compara dos piezas y se calcula por separado. No hay orden entre ellas.

¿RECUPERÓ BIEN? • MIRA EL CONTEXTO

Context precision

nota 0 a 1

compara:

Pregunta

Contexto

¿lo que recuperó era relevante, sin ruido?

Context recall

nota 0 a 1

compara:

Contexto

Referencia

¿el contexto trae todo lo necesario?

¿GENERÓ BIEN? • MIRA LA RESPUESTA

Faithfulness

nota 0 a 1

compara:

Respuesta

Contexto

¿la respuesta se apoya en el contexto?

Answer relevancy

nota 0 a 1

compara:

Respuesta

Pregunta

¿la respuesta va al grano de la pregunta?

Si una nota sale baja, ahí está el problema: en el contexto (recuperación) o en la respuesta (generación).

Faithfulness: ¿se apoya en el contexto?

Faithfulness revisa si cada afirmación de la respuesta se puede verificar en el contexto recuperado. Una respuesta correcta pero que no sale del contexto (el modelo ya la sabía) puntúa bajo, y eso es una alerta: en producción, con otro contexto, podría alucinar.



Correctitud factual

Es la métrica que más atrapa alucinaciones: no mira si la respuesta suena bien, sino si el contexto la respalda.

¿El contexto trae lo justo, y todo?

CONTEXT PRECISION



¿Poco ruido?

De lo que recuperamos, ¿qué proporción era de verdad relevante? Baja si el top-k trae chunks que no ayudan. La sube el reranking de la S08.

CONTEXT RECALL



¿Está todo?

¿El contexto trae toda la información necesaria para la respuesta esperada? Baja si el retrieval dejó afuera un chunk clave. La suben hybrid o query transformation.

Cada técnica de la S08 mueve una de estas dos: por eso las medimos por separado.

Se complementan, no se reemplazan.

Answer relevancy mide si la respuesta va al grano, sin rodeos ni relleno. Y algo importante: RAGAS no reemplaza a nuestro harness, lo complementa. El harness dice si acertamos; RAGAS dice por qué (retrieval o generación). Los reportamos juntos en la entrega.

Nota práctica: RAGAS necesita un LLM y embeddings para calcular las métricas. En el lab lo vemos por dentro con nuestro propio juez, y dejamos lista la librería `ragas` para el reporte de la entrega.

Al notebook · Lab C

Evaluar el sistema con RAGAS

Preparamos, por cada caso, la pregunta, la respuesta, el contexto recuperado y la referencia. Calculamos faithfulness, context precision, context recall y answer relevancy, y las leemos junto a nuestro scorecard propio. Vemos qué métrica delata cada tipo de fallo.

```
dataset(pregunta, respuesta, contexto, referencia) → RAGAS → 4 métricas
```

Qué nos dijo RAGAS.

- **Faithfulness levantó una alerta** donde la respuesta no salía del contexto, la métrica lo marcó aunque el texto sonara bien.
- **Precision y recall separan el fallo** vimos si el problema era ruido en el contexto o un chunk que faltó.
- **Junto al harness, cuadra** las dos vistas juntas dicen qué arreglar: retrieval o generación.

No pierdan las evaluaciones: guárdenlas.

Cada vez que corren el harness o RAGAS obtienen números que conviene no perder. Weights & Biases es la herramienta de seguimiento del curso: registran ahí los puntajes de cada corrida, la latencia y, si quieren, la traza del agente (qué herramienta llamó y en qué paso). Así comparan versiones de su sistema en el tiempo, en vez de anotar números sueltos.

Es opcional para la entrega, pero vale la pena tenerlo desde ya: en la S15 W&B se vuelve central, cuando instrumentamos el sistema desplegado con trazas, latencia y errores en producción.

Cerramos el módulo.

El sistema completo, evaluado, y qué entregamos hoy.

El sistema completo, de principio a fin.



Recuperación avanzada de la S08, más herramientas cuando hacen falta, más evaluación con RAGAS y el harness.

Ese es el sistema que entregan hoy: no solo que funcione, sino que sepan **por qué** funciona y qué cuesta.

Qué entra en la entrega.

1

El sistema RAG completo, con al menos dos técnicas avanzadas y al menos una herramienta (tool).

2

El reporte de evaluación: su scorecard propio más las métricas de RAGAS.

3

Una lectura honesta: qué técnica movió qué, qué costó en latencia y qué falla queda pendiente.

Se entrega 27 SEPT. Es el 15% del curso.

¿Y si el agente recuerda?

MEMORIA CONVERSACIONAL



Seguir una conversación

Guardar el contexto entre turnos deja mantener un hilo, no solo responder preguntas sueltas. El asistente recuerda lo que ya se dijo.

EL COSTO



No es gratis

Cada turno arrastra más tokens (más caro y lento), y el contexto viejo puede desviar la respuesta. A eso se le llama deriva.

Útil para asistentes con conversación. Fuera del alcance del curso, pero bueno saberlo.

Optimizar el prompt, no solo escribirlo.

Hasta ahora ajustamos los prompts a mano. Pero una vez que tienen una métrica (su harness o RAGAS), pueden dejar que una herramienta busque un mejor prompt de forma automática, midiendo cada intento contra esa métrica.

DSPy es la más conocida: en vez de escribir el prompt, describen la tarea y dan unos ejemplos, y DSPy arma el prompt (instrucciones y ejemplos) que maximiza el puntaje.

No lo cubrimos a fondo en el curso, **pero es la continuación natural de tener una buena evaluación: sin una métrica confiable no vale la pena optimizar el prompt; con ella, se puede automatizar.**

La próxima unidad: visión.

Con esto cerramos el módulo de RAG. La S11 abre la Unidad 4: llevamos el transformer a las imágenes (ViT, CLIP) y a lo multimodal (LLaVA). Cada equipo sumará un componente visual a su sistema en la próxima entrega.

Recuerden subir su entrega ANTES DEL 27 SEPT 11:59 PM: el sistema RAG completo más el reporte con el scorecard propio y las métricas de RAGAS.

Ahora el RAG actúa cuando hace falta,
y sabemos **evaluarlo por dentro.**

Tool use y ReAct le dan acciones; RAGAS mira el contexto, no solo la respuesta. Un agente se justifica solo si el harness lo confirma.